



Guidewire PolicyCenter™

Integrating Products with PolicyCenter

Release: 2026.03



© 2026 Guidewire Software, Inc.

For information about Guidewire trademarks, visit <https://www.guidewire.com/legal-notices>.

Guidewire Proprietary & Confidential — DO NOT DISTRIBUTE

Product Name: Guidewire PolicyCenter for Guidewire Cloud

Product Release: 2026.03

Document Name: Integrating Products with PolicyCenter

Document Revision: 01-April-2026

Contents

Support.....	5
1 Getting started.....	7
Request to enable APD App.....	7
Set the working context.....	7
2 Integrate APD App with PolicyCenter.....	9
Move products from APD in PolicyCenter to APD App.....	9
Before you begin.....	9
Instructions.....	9
Next steps.....	11
Related information.....	11
Visualizing APD App products.....	11
Synchronizing PolicyCenter with the APD cloud service.....	11
Creating a submission in PolicyCenter.....	12
Manage typelists.....	12
Use a PolicyCenter typelist to provide values for a drop-down list.....	12
Language translations.....	12
Create required instances of the APD service.....	13
Related information.....	13
Test products in a local PolicyCenter environment.....	14
Configure local PolicyCenter environment to visualize products.....	14
Write unit tests in a local PolicyCenter environment.....	16
Move to a new release of PolicyCenter.....	19
Regenerate an insurance product.....	19
Create a typelist extension.....	19
3 Finalizing and generating the product.....	21
Metadata.....	22
Adding Short Name (*PD).....	23
Add metadata.....	24
Review product content.....	25
Validated fields.....	25
Validate product in Designer mode.....	25
Validate product in Developer mode.....	26
Move product.....	26
Generating and deploying a product.....	27
Edit PolicyCenter settings.....	29
Define Gosu package for product line files.....	29
Generate product code.....	30
Deploy product on Guidewire Cloud.....	31
Regenerating product code.....	31
Regenerating the product.....	32
Installed product files.....	33
Variations in installed products.....	40
Updating insurance products.....	41

Support

For assistance, visit the Guidewire Community.

Guidewire customers

<https://community.guidewire.com>

Guidewire partners

<https://partner.guidewire.com>

Getting started

Request to enable APD App

You must [Request to enable Advanced Product Designer](#) to create a ticket to provision APD App.

You must add the appropriate access groups to your IdP and confirm that you have access to Guidewire Home. `gwa.any_planet_class.apd.designer` must be configured in your IdP to support APD.

Because APD depends on the Lifecycle Manager (LCM), the following groups must be configured:

- Read-only: `gwa.any_planet_class.lifecyclemanager.readonly`
- All: `gwa.any_planet_class.lifecyclemanager.admin`

Prior to deploying services dependent on LCM in production or non-production environments, LCM must have a configuration deployed on that planet. Assign an LCM configuration to planets to avoid errors that might occur during runtime.

Set the working context

Before using Advanced Product Designer App to create an insurance product, you must set the working context.

For more information about setting the working context, see *Guidewire Home*.

Integrate APD App with PolicyCenter

Move products from APD in PolicyCenter to APD App

Move insurance products that were created in Advanced Product Designer in PolicyCenter from locally-managed to cloud-managed. This is a great way to take full advantage of functionality available in the cloud. After moving an insurance product, you must make any changes in APD App.

After reading this topic, you will be able to:

- Export the XML template for the locally-managed product. If you do not have the current version of the template, contact your Guidewire representative.
- Import the product into APD App.
- Sync and codegen (install) the product.

For a demonstration, watch this video:

Before you begin

1. Identify insurance products that were created in Advanced Product Designer in PolicyCenter.
2. Decide which products you want to move from locally-managed to cloud-managed.
3. For each product, export the XML template for the locally-managed product, import the product into APD App, and resolve any errors.

In some cases, when a rule has been added to a product and then removed, an orphan `<rules>` tag appears in the XML template. Remove the empty tag to resolve the error.

If you cannot resolve an error, open a case and include the error received, the date and time when you attempted the import, and the XML template.

Instructions

Step 1: Export the XML template for the locally-managed product

In PolicyCenter, select the product you want to move and export the XML template.

1. In PolicyCenter, on the **Product Management** screen, on the **APD Managed** tab, select the product you want to move. The **Source** column shows **Local**, indicating that the product is locally-managed.

Product Management

APD Managed | Externally Managed

Import From Meta | Import From Template | Import From Cloud | Synchronize With Cloud | New Product

Product Name	Product Description	Installed	Date Installed	Last Updated	Source
GO Personal Auto	GO Personal Auto			10/13/2022 3:18 PM	Cloud
Cyber Product	Cyber Product			10/13/2022 3:18 PM	Cloud
Quantum Watercraft 2.0	Quantum Watercraft 2.0			10/13/2022 3:18 PM	Cloud

- Click **Export Template** to download the XML template.

Step 2: Import the product into APD App

In APD App, create a new insurance product using the XML template you downloaded earlier.

- In APD App, click **Create New Product**.
- In **Load Product**, on the **From Template** tab, click **Upload XML file**.
- Upload the XML template you downloaded earlier.
- Click **Create**. After a few seconds, a tile for the product appears on the **Insurance Products** screen. You can now modify the product in APD App.

Insurance Products

GO Personal Auto Checked out by someone else URL: /app Updated Oct 13, 2022, 3:18 AM GO Personal Auto	Cyber Product Available CMP: /app Updated Oct 13, 2022, 3:18 AM Cyber Product	Quantum Watercraft 2.0 Available TPL: /app Updated Oct 13, 2022, 3:18 PM Quantum Watercraft 2.0
--	--	--

Step 3: Sync and codegen (install) the product

In PolicyCenter, import the insurance product from APD App and generate the code.

- In PolicyCenter, on the **Product Management** screen, on the **APD Managed** tab, click **Remove Product** to remove the visualized product, as you cannot have two visualized products with the same name. Click **OK** to confirm.
- Click **Synchronize With APD** to import the product from APD App. The **Source** column shows **Cloud**, indicating that the product is cloud-managed.
- Select the product and click **Generate Product Code**.
- On the **Review Product Elements** screen, in **Branch Suffix**, type the name of the Bitbucket branch in which to install the product. The branch name must be unique.
- Select the **Generate Options**.
 - For **Base Product**, select **Default Directory**.
 - For **System APIs - Code**, it is recommended to select **Default Directory**.
 - Generating **Rating Files**, **Page Configuration Files**, and **System APIs - Tests** is optional. Select **Default Directory** or **Temporary Directory** to generate these files, or select **Do Not Generate**.
- Click **Complete Generation** and confirm. After a few seconds code generation is complete.
- Click **Return to Product Management**. Today's date appears in the **Date Installed** column, indicating that the product is installed.

Product Management

APD Managed | Externally Managed

Import From Meta | Import From Template | Import From Cloud | Synchronize With Cloud | New Product

Product Name	Product Description	Installed	Date Installed	Last Updated	Source
GO Personal Auto	GO Personal Auto			10/13/2022 3:57 PM	Cloud
Cyber Product	Cyber Product			10/13/2022 3:57 PM	Cloud
Quantum Watercraft 2.0	Quantum Watercraft 2.0		10/13/2022	10/13/2022 3:57 PM	Cloud

Step 4: Test and promote the product to production

Review the build logs, test the code locally, and promote the product to production.

- In Guidewire Studio, review the updated product code.

2. Restart PolicyCenter in Guidewire Studio and review the start and build logs until all errors are resolved.
3. Validate and test the product code locally.
4. Promote and deploy the product to production.

After code generation

After generating the code, take note of the following:

- It is no longer necessary to load and activate editions. Editions are provided through the APD service.
- Product synchronization loads the product metadata from APD App.
- All product configuration must be done in APD App.

Next steps

Now you can modify the insurance product in APD App. You can take full advantage of functionality available in the cloud such as product segmentation and multi-line.

- “Visualizing APD App products” on page 11
- “Finalizing and generating the product” on page 21
- “Regenerating product code” on page 31

Related information

- *PolicyCenter Application Guide*

Visualizing APD App products

Visualizing your APD App product allows you to interact with your insurance product in PolicyCenter.

Complete the following steps to visualize your APD App product:

1. **Optional:** If you are working in a local installation of PolicyCenter, you must first configure your local environment. See “Configure local PolicyCenter environment to visualize products” on page 14.
2. Synchronize PolicyCenter with the APD App product.
3. Create a submission in PolicyCenter.

Synchronizing PolicyCenter with the APD cloud service

It’s easy to fetch APD App products and synchronize them with PolicyCenter. Synchronizing PolicyCenter with APD cloud service ensures that changes made in APD App are also present in the visualized product.

When PolicyCenter makes a request to APD App, APD App identifies the products configured for PolicyCenter’s environment. These are the products you can fetch or synchronize. You have several options for fetching or synchronizing APD App products from within PolicyCenter:

- **Synchronize With APD**, located on the **Product Management** screen, synchronizes the product definitions for all products with the APD cloud service.
- **Import From APD**, located on the **Product Management** screen, loads the product with the specified identifier from the cloud.
- **Update Product > From APD**, located on the **Details** tab on the **Product Management** screen, updates the selected product with any product model changes from the cloud.
- Setting the Property Service property `product.reload.enabled` to true automatically refreshes the products periodically. `product.reload.enabled` is located in the `pc.apd` namespace.

Note: To remove a product, you must remove it from both APD App and the **Product Management** screen in PolicyCenter. If you do not remove the product from APD App, it reappears when you next synchronize with the cloud.

For a short demonstration, watch this video:

Creating a submission in PolicyCenter

Visualize APD insurance products by creating a submission in PolicyCenter.

After an APD App product is synchronized in PolicyCenter, you can create a submission in PolicyCenter to verify that the product functions as expected. For instructions on how to create a submission in PolicyCenter, see the *PolicyCenter Application Guide*.

Note: If you are working in a non-production environment, you must regenerate the code and redeploy the product to a planet after adding an element such as a field, coverage, or term in APD App and creating a rule for the element. If you do not regenerate the code and redeploy, PolicyCenter throws a `NullPointerException` when you attempt to start a submission.

Manage typelists

APD fetches values from currency, language, rate factor, and other externally-managed typelists in PolicyCenter.

When you invoke the **Manage Currencies** or **Manage Languages** dialog, APD takes a few seconds to fetch values from the appropriate typelist in PolicyCenter. Values fetched from a PolicyCenter typelist are marked with an asterisk (*). Any values you added in APD also appear.

When you invoke the **Manage Rate Factor Types** dialog, APD lists the rate factor types. In the **External** column, **Yes** indicates that the value is provided by PolicyCenter and **No** indicates that the value has been added in APD. For instructions on how to add a rate factor type, see *Creating Products with APD App*.

Use a PolicyCenter typelist to provide values for a drop-down list

You can use a PolicyCenter typelist as an externally-managed typelist in APD, for example to provide values for a drop-down list. The names and definitions of PolicyCenter typelists are available in Guidewire Studio.

Procedure

1. In **Drop-down definition**, in **Drop-down info**, in **Select List**, type the name of a typelist or choose from the drop-down list.
2. Check **Externally Managed and Referenced**.
3. Choose **Sync**.
The values in the PolicyCenter typelist appear in **Drop-down items**. A **Yes** in the **External** column indicates that the value is provided by an externally-managed typelist.
4. Click **Save**.

Language translations

The default language in APD App is the primary language used to define product elements, for example in the **Product Name** and **Product Description** fields at the top of the **Element Details** pane. Translations for other languages may be provided under **Localization** at the bottom of the **Element Details** pane.

When you synchronize an APD product to PolicyCenter, translations are filled based on the default language and translations provided in the product model in APD App as well as the default language and other languages configured in PolicyCenter. (The language in which the PolicyCenter user interface is displayed is not taken into consideration.) If any translations are missing, PolicyCenter displays messages indicating that the default values provided in the product model in APD App will be used.

If no language is explicitly set for the values of localizable elements, "fallback" values are used. The configuration parameter `APDCodegenApplicationLanguage` specifies the language for the fallback values when importing products from APD and generating code for these products. If `APDCodegenApplicationLanguage` is not set, the language specified by the configuration parameter `DefaultApplicationLanguage` is used.

Example: Only the default language is used

In APD App, only the default language is used. No additional translations are provided.

In PolicyCenter, English is the default language.

When the product is synchronized to PolicyCenter, the default language used in the product is mapped to the default language configured in PolicyCenter. No other translations are filled.

Example: Two additional translations are provided

In APD App, two additional translations are provided: Spanish and German.

In PolicyCenter, English is the default language. The Spanish language is configured, but the German language is not.

When the product is synchronized to PolicyCenter, the default language used in the product is mapped to the default language configured in PolicyCenter. Spanish translations are filled while German translations are ignored.

Example: Three additional translations are provided

In APD App, three additional translations are provided: Spanish, German, and Japanese.

In PolicyCenter, Spanish is the default language. The German language is configured, but the Japanese language is not.

When the product is synchronized to PolicyCenter, Spanish and German translations are filled while Japanese translations are ignored.

Create required instances of the APD service

The APD service plugin, `IApdServicePlugin`, controls the integration between PolicyCenter and the APD service (APDS). Use `IApdServicePlugin` to create the required instances of the APD service.

You typically need to create three instances of the APD service as shown in the following table. The instance of the service is determined by an environment variable. You add these environment variables in Guidewire Studio.

Note: The environment variables may vary for your implementation.

Service	Environment variable	Usage
Mock	h2mem	Run GUnits in h2mem during Server Tests in TeamCity. If the tests do not execute, set the environment variable to <code><null></code> .
Local	local, empty, or per configuration	Connect to the APD service locally.
OOTB	cloud-dev	Connect to the APD service in a cloud environment.

Related information

- *PolicyCenter Plugins, Prebuilt Integrations, and SOAP APIs*

Test products in a local PolicyCenter environment

Configure local PolicyCenter environment to visualize products

Using a local installation of PolicyCenter, you can test policy transactions that call editions from APD App. You must use a development version of the APD service plugin and authenticate against the APD service using a token.

Make sure each user is a member of the correct Active Directory user groups.

Table 1: Non-APD BAs, developers, or testers using a local instance of PolicyCenter

Application	User group
Guidewire Home	<code>gwa.any_planet_class.gch.user</code>
APD App	<code>gwa.any_planet_class.apd.designer</code>

Table 2: BAs or developers using APD App or performing code generation/build/deploy

Application	User groups
Guidewire Home	<code>gwa.any_planet_class.gch.user</code> <code><tenant>.dev.<starsystem>.all.gcc.developers</code>
Bitbucket	<code><tenant>.dev.<starsystem>.all.scm.developers</code>
TeamCity	<code><tenant>.dev.<starsystem>.all.ci.developers</code>
APD App	<code>gwa.any_planet_class.apd.designer</code>
Lifecycle Manager	<code>gwa.any_planet_class.lifecyclemanager.admin</code>
Datadog	<code><tenant>.dev.<starsystem>.all.log.developers</code>

Use the development version of the APD service plugin

These settings are for local development only. Do not use these settings for any other purpose as they negatively impact cloud deployments.

Before you begin

You must set several properties in the Server configuration in Guidewire Studio. Choose **Run > Edit Configurations** and add these environment variables:

- `GW_TENANT` is the name of the tenant.
- The setting of `DEPLOYMENT_ID` is of the form `grn:version:app:tenant:project:galaxy:quadrant:planetClass:planet:/resourceId`, for example `grn:v1:pc:guidewire:salesdemo:centaurus:centaurus:dev:myplanet:/deployment/myplanet`. The planet from this GRN is used to call planet-based endpoints.
- `GW_PROPERTY_SERVICE_DISABLED` is set to `true`.

Use the PolicyCenter values from Guidewire Home to determine the values for `GW_TENANT` and `DEPLOYMENT_ID`. For more information see *Deploy InsuranceSuite Applications*.

About this task

`ApdServicePluginLocalDevImpl` is an implementation of `APDServicePlugin` that enables you to connect to APD App from a local installation of PolicyCenter using a token.

Note:

- In most cases, you can use the development version of the APD service plugin without installing additional certificates. However, in rare cases, you may need to install certificates such as Okta and APD Service in

your local environment in order to communicate with Guidewire services. Use the `keytool` command to install certificates, for example

```
keytool -importcert -trustcacerts -storepass changeit -alias okta.com.crt  
-keystore <JAVA_HOME>\lib\security\cacerts -file <location of the downloaded cert>
```

- Because LCM is disabled, editions are not automatically refreshed. If you make any changes to editions in APD App, update the product from the cloud to pull the most recent versions.
- `ApdServicePluginLocalDevImpl` must not be used in a deployed environment.

Procedure

1. For `IApdServicePlugin`, use `com.guidewire.pc.cloud.apd.ApdServicePluginLocalDevImpl` as the implementation.
2. In `config.properties`:
 - a) Enable APD App by setting `plugin.apd.service.disabled=false`.
 - b) Specify the APD URL by setting `plugin.apd.service.baseUrl=MyAPDInstance`, where `MyAPDInstance` is the URL of the APD instance you are using. Use the URL for the region in which your tenant is provisioned:
 - For AMER (including older Canada deployments), <https://apd.api.omega2-andromeda.guidewire.net/>
 - For APAC (including older Japan deployments), <https://apd.api.omega2-circinus.guidewire.net/>
 - For EMEA, <https://apd.api.omega2-cartwheel.guidewire.net/>
 - For Canada (new deployments), <https://apd.api.omega2-butterfly.guidewire.net/>
 - For Japan (new deployments), <https://apd.api.omega2-milkyway.guidewire.net/>
 - For Zurich's Region, Whirlpool EU-CENTRAL-2 (Zurich) EMEA-CH, <https://apd.api.omega2-whirlpool.guidewire.net/>

Authenticate against the APD service using a token

Before you begin

- Set **Product Design Mode** to **Developer**.

About this task

Copy a token from APD App and paste it into PolicyCenter to authenticate against the APD service. After authenticating, you can import cloud products and test policy transactions.

Procedure

1. In APD App, on the **Insurance Products** screen, click **Get APD Dev Token**.
2. In Authentication Token, click **Copy**.

Take note of how much time remains before the token expires. After the token expires, PolicyCenter throws an exception when you attempt to retrieve a cloud product. You must get another token before continuing.
3. In PolicyCenter, on the **Product Management** screen, click **Authenticate With APD**.
4. On the **Authenticate with APD using token** screen, in **Token**, paste the token. Click **Update**.
5. On the **Product Management** screen, click **Synchronize With APD** or **Import From APD** to retrieve the product.
6. Test a submission and other policy transactions as necessary.

Write unit tests in a local PolicyCenter environment

You can validate an installed product and its editions using a local APD service plugin that implements the interface `IAPDServicePlugin`. You cannot use this approach with a visualized product.

You use the APD API to generate JSON responses that are saved to local JSON files. If you do not have access to the API, create a ticket requesting the following:

- Client ID and secret with the `apds.products.read` scope
- Access to the APD API

The reference implementation takes the JSON files and converts them to DTOs.

Reference implementation for unit tests

This reference implementation fetches the JSON responses from the current active product.

Each calling method takes a JSON file and parses the object or list of objects it contains. The plugin parses the following files:

- `getAllProductSummaries.json`
- `getProduct-[PRODUCT_CODE].json`
- `getAllEditionSummaries-[PRODUCT_CODE].json`
- `getEdition-[PRODUCT_CODE]-[EDITION_CODE].json`
- `getConfiguredSegments.json`

Export the product from APD App as a ZIP file and rename the editions and product files to comply with `ExampleAPDMockedServicePluginImpl`. For an example, see `SimpleCyberCrime-precanned-jsons.zip`.

You can generate the JSON for other lines of business using these responses as an example. Any new LOBs must be added to the `getAllProductSummaries.json` file. If you want to reference another LOB in PolicyCenter, all JSON files related to that product must be accessible by `ExampleAPDMockedServicePluginImpl`. For example, to import an LOB with product code `InlandMarine` from the cloud, make sure that you have all the JSON files that have `InlandMarine` in their name and an entry in `getAllProductSummaries.json`.

IAPDServicePlugin.gwp

```
<plugin
  interface="IAPDServicePlugin"
  name="IAPDServicePlugin">
  <plugin-gosu
    gosuclass="gw.plugin.ExampleAPDMockedServicePluginImpl"/>
  </plugin>
```

Sample plugin that maps JSON to DTOs

```
package gw.plugin

uses com.fasterxml.jackson.databind.DeserializationFeature
uses com.fasterxml.jackson.databind.ObjectMapper
uses com.fasterxml.jackson.databind.SerializationFeature
uses com.fasterxml.jackson.databind.util.StdDateFormat
uses com.fasterxml.jackson.datatype.jsr310.JavaTimeModule
uses gw.apdcloud.client.model.APDEditionDTO
uses gw.apdcloud.client.model.APDProductDTO
uses gw.apdcloud.client.model.EditionSummary
uses gw.apdcloud.client.model.ProductSummary
uses gw.apdcloud.client.model.Segment
uses gw.api.productedition.ActiveEditionResult
uses gw.api.util.ConfigAccess
uses gw.plugin.apd.IAPDServicePlugin

uses java.io.File
uses java.text.DateFormat
uses java.text.FieldPosition
uses java.text.ParsePosition
uses java.time.OffsetDateTime
```



```

@Export
class ExampleAPDMockedServicePluginImpl implements IApdServicePlugin {

    // cache the data
    private static var _productSummariesData : List<ProductSummary>
    private static var _productData = new HashMap<String, APDProductDTO>()
    private static var _editionSummariesData = new HashMap<String, List<EditionSummary>>()
    private static var _editionData = new HashMap<String, APDEditionDTO>()
    private static var _segmentData : List<Segment>
    private static var _activeEditionData = new HashMap<APDEditionDTO, ActiveEditionResult>()

    private var savedJsonRootDir = ConfigAccess.getModuleRoot("configuration")?.CanonicalPath
        + "/config/content/apdartifacts/mockapd/"
    private var _objectMapper : ObjectMapper

    construct() {
        _objectMapper = new ObjectMapper()
        _objectMapper.enable(SerializationFeature.WRITE_ENUMS_USING_TO_STRING);
        _objectMapper.enable(DeserializationFeature.READ_ENUMS_USING_TO_STRING);
        _objectMapper.disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
        _objectMapper.disable(DeserializationFeature.FAIL_ON_INVALID_SUBTYPE);
        _objectMapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        _objectMapper.setDateFormat(new RFC3339DateFormat());
        _objectMapper.registerModule(new JavaTimeModule());
    }

    override function getProduct(productCode : String, packageUid : UUID) : APDProductDTO {
        var filename = "getProduct-${productCode}.json"
        return _productData.computeIfAbsent(filename,
            \k -> getObjectFromFile<APDProductDTO>(filename))
    }

    override function getEdition(productCode : String, packageUid : UUID, editionCode : String)
        : APDEditionDTO {
        var filename = "getEdition-${productCode}-${editionCode}.json"
        return _editionData.computeIfAbsent(filename,
            \k -> getObjectFromFile<APDEditionDTO>(filename))
    }

    override property get AllProducts() : List<ProductSummary> {
        if (_productSummariesData == null) {
            _productSummariesData = getListOfObjectFromFile<ProductSummary>
                ("getAllProductSummaries.json")
        }
        return _productSummariesData
    }

    override function getAllEditions(productCode : String) : List<EditionSummary> {
        var filename = "getAllEditionSummaries-${productCode}.json"
        return _editionSummariesData.computeIfAbsent(filename,
            \k -> getListOfObjectFromFile<EditionSummary>(filename))
    }

    override function getAllEditionDetails(productCode : String) : List<APDEditionDTO> {
        var summaries = getAllEditions(productCode)
        return summaries.map(\elt -> getEdition(productCode, null, elt.Code))
    }

    override function getActiveProduct(productCode : String) : APDProductDTO {
        return getProduct(productCode, null);
    }

    override function getActiveEdition(productCode : String, referenceDate
        : OffsetDateTime, jurisdiction : Jurisdiction) : APDEditionDTO {
        var activeEditionCode = getAllEditions(productCode)
        // add searching logic here if needed
        // .where(\elt -> elt.EffectiveDate.isBefore(referenceDate) and ...)
        .first().Code

        var filename = "getEdition-${productCode}-${activeEditionCode}.json"
        return _editionData.computeIfAbsent(filename,
            \k -> getObjectFromFile<APDEditionDTO>(filename))
    }

    override function getActiveEditionResult(activePackageUidAtEditionStoredTime : UUID, productCode
        : String, referenceDate : OffsetDateTime, jurisdiction : Jurisdiction) : ActiveEditionResult {
        var activeEdition = getActiveEdition(productCode, referenceDate, jurisdiction)
        return _activeEditionData.computeIfAbsent(activeEdition,
            \k -> new ActiveEditionResult(activeEdition, null, new Date(referenceDate.toInstant().toEpochMilli()),
                jurisdiction))
    }

    override function getActiveEditionResult(activePackageUidAtEditionStoredTime : UUID, productCode
        : String, referenceDate : OffsetDateTime, jurisdiction : Jurisdiction, segments : List<String>)
        : ActiveEditionResult {
        var activeEdition = getActiveEdition(productCode, referenceDate, jurisdiction)
        return _activeEditionData.computeIfAbsent(activeEdition,
            \k -> new ActiveEditionResult(activeEdition, null, new Date(referenceDate.toInstant().toEpochMilli()),
                jurisdiction, segments))
    }
}

```

```

    }

    override property get ConfiguredSegments() : List<Segment> {
        if (_segmentData == null) {
            _segmentData = getListOfObjectFromFile<Segment>("getConfiguredSegments.json")
        }
        return _segmentData
    }

    private reified function getObjectFromFile<T>(jsonFilename : String) : T {
        var jsonString = getFileContents(jsonFilename)
        return _objectMapper.readValue(jsonString, T)
    }

    private reified function getListOfObjectFromFile<T>(jsonFilename : String) : List<T> {
        var jsonString = getFileContents(jsonFilename)
        var jsonNode = _objectMapper.readTree(jsonString);
        return jsonNode.get("data").map(\elt -> _objectMapper.readValue(elt.toString(), T))
    }

    private function getFileContents(jsonFilename : String) : String {
        return new File(savedJsonRootDir + jsonFilename).read();
    }

    override property set Parameters(map : Map<Object, Object>) {
    }

    private class RFC3339DateFormat extends DateFormat {
        private static final var serialVersionUID = 1L
        private final var TIMEZONE_Z = TimeZone.getTimeZone("UTC")
        private final var fmt = new StdDateFormat().withTimeZone(TIMEZONE_Z).withColonInTimeZone(true)

        construct() {
            this.calendar = new GregorianCalendar()
        }

        override function parse(source : String) : Date {
            return parse(source, new ParsePosition(0))
        }

        override function parse(source : String, pos : ParsePosition) : Date {
            return fmt.parse(source, pos)
        }

        override function format(date : Date, toAppendTo : StringBuffer, fieldPosition : FieldPosition) : StringBuffer {
            return fmt.format(date, toAppendTo, fieldPosition);
        }

        override function clone() : Object {
            return this
        }
    }
}

```

Use the plugin in tests

The following example uses the JSON-backed plugin in tests. Using `registerPlugin` updates the test server to use the plugin. In this example we use the custom plugin for the duration of the test by setting it in `beforeClass` and resetting it back in `afterClass`. When the test is complete, it resets the plugin back to the original plugin so as not to interfere with any subsequent tests.

```

class UsingExampleAPDMockedPluginTest extends PCServerTestClassBase {
    private static var _originalApdPlugin : IApdServicePlugin

    override function beforeClass() {
        super.beforeClass()

        _originalApdPlugin = Plugins.get(IApdServicePlugin);

        var exampleAPDMockedServicePlugin = new ExampleAPDMockedServicePluginImpl()
        registerPlugin(IApdServicePlugin, exampleAPDMockedServicePlugin)
    }

    override function afterClass() {
        super.afterClass()
        registerPlugin(IApdServicePlugin, _originalApdPlugin);
    }
}

```

Alternatively, add a new implementation in `IAPDServicePlugin.gwp` and set `h2mem` as the environment for the running instance to ensure that the mock service only runs for Gunits. This approach is valid as long as the TeamCity environment uses `h2mem` for the server tests and the environment variable is respected when the Gunits run.

Move to a new release of PolicyCenter

When you move to a new release of PolicyCenter, to access new features in APD App, open a ticket with Guidewire Customer Operations requesting an update to the `apd.pc.releaseVersion` property.

Then, move each cloud insurance product to the new release by doing one of the following:

- Regenerate the insurance product.
- Create a typelist extension for the insurance product.

You only need to regenerate or create a typelist extension once. This step will not be necessary the next time you move to a new release of PolicyCenter.

Note: For local insurance products, it is not necessary to regenerate or create a typelist extension.

Regenerate an insurance product

Procedure

1. In PolicyCenter, set **Product Design Mode to Developer**.
2. Navigate to **Products > Product Management**, and select the product you want to generate.
3. Generate the product code by selecting the product and clicking **Generate Product Code**.
4. Merge code changes in Bitbucket.

See also

- “Regenerating the product” on page 32
- “Installed product files” on page 33

Create a typelist extension

Procedure

1. Create a typelist extension `APDCloudProduct.xxx.ttx`, where `xxx` is the product abbreviation specified in APD App, in the `PolicyCenter/modules/configuration/config/extensions/typelist` folder. For example, for a commercial inland marine product, create a typelist extension called `APDCloudProduct.IMC.ttx`.
2. Add a single entry that contains the code, description, and name of the product specified in APD App, for example `<typecode code="MyCommInlandMarine" desc="Commercial Inland Marine" name="MyCommInlandMarine"/>`.

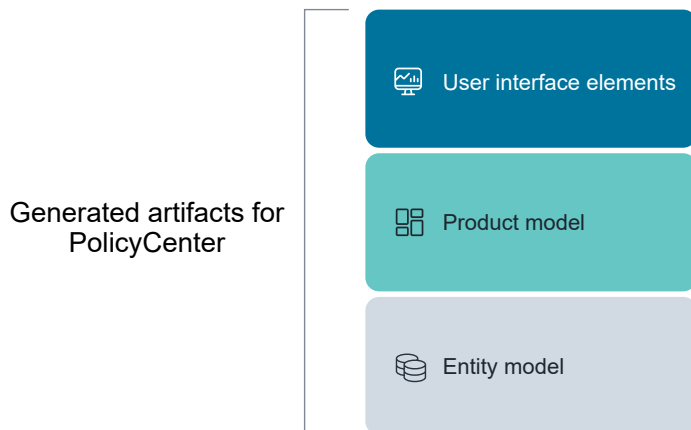
The complete typelist extension for the commercial inland marine product, `APDCloudProduct.IMC.ttx`, is as follows:

```
<?xml version="1.0"?>
<typelistextension
  xmlns="http://guidewire.com/typelists"
  name="APDCloudProduct">
  <typecode code="MyCommInlandMarine" desc="Commercial Inland Marine" name="MyCommInlandMarine"/>
</typelistextension>
```


Finalizing and generating the product

About this task

When you have refined and extended the visualized product, you are ready to finalize the product. Finalization is the phase where the visualized product is used to generate the artifacts needed for PolicyCenter including user interface elements, product model, and entities. The result is an installed product.



The result of finalization is an installed product that has a check in the **Installed** column on the **APD Managed** tab of the **New Submissions** screen in PolicyCenter.

Save frequently during finalization

Use **Export Template** on your product prior to the final build.

Note: If you drop your database before the final build is complete, you will lose your work on the visualized product.

Parts of visualized product not in installed product

- The rate in pricing
- Event data

Develop features not provided in Advanced Product Designer App

For features not provided for in Advanced Product Designer App, continue development using tools provided by PolicyCenter and described in the PolicyCenter documentation. Complete development by adding underwriting rules and rating, event data, integrating with other applications, and making other changes to meet your needs.

Continue product design

You can continue to enhance the product in Advanced Product Designer App as described in “Updating insurance products” on page 41.

Procedure

1. Review all metadata codes and descriptions to ensure that they follow Guidewire suggested protocols, or protocols agreed upon by your the project. See “Metadata” on page 22.
2. Review all content. See “Review product content” on page 25.
3. In the PolicyCenter installation where you will generate the product code, set the package name for installed product line Gosu files in `APDProductLinesGosuPackage` script parameter.
4. Validation reviews the product definition for completeness and correctness. Validation can be done in any PolicyCenter instance where business users create products.
 - a) Business user starts validation of the product definition. See “Validate product in Designer mode” on page 25.
 - b) Developer completes validation of the product definition. See “Validate product in Developer mode” on page 26.
5. (Optional) When the product is valid, save it as an APD template and check it into version control.
6. (Optional) If it is not already there, move the product to the PolicyCenter instance where it will be generated and deployed. See “Move product” on page 26.
7. Generate, build, and deploy the product. See “Deploy product on Guidewire Cloud” on page 31.

You generate the product in the PolicyCenter instance where you continue development. Product generation accesses and saves files in the PolicyCenter installation. The product may need to be regenerated after making changes.

Metadata

You must add metadata for all elements in the product model tree in fields labeled **(*PD)**. These fields are not required to visualize the product, but are required for product generation.

The metadata includes business descriptions and codes, such as:

- General product details
- Line details and risk objects
 - Fields
 - Coverages, conditions, and exclusions
 - Drop-down lists associated with fields, coverages, conditions, and exclusions

The product model is not valid until all metadata has been entered.

How to specify metadata

You can specify metadata for the product model by:

- Entering **(*PD)** values in the visualized product screens directly.
- Defining short names in the mind map.

Adding Short Name (*PD)

Short names are easily readable codes that are used as internal identifiers in PolicyCenter, for example when generating PCFs. Create short names that follow best-practice naming conventions agreed upon by the organization or team.

The APD App product screens have **Short Name** fields. These fields must be filled in before generating the product. For drop-down lists, you specify the short name in the **Typelist** field.

Mind map label for short name

For all topics with markers (except Product ❖ and Product Line 🏠) in the XMind mind map, you can specify the short name in a label. The label becomes the **Short Name** in the visualized product.

Short name requirements

Short names must:

- Start with an uppercase letter
- Include only letters, digits, \$ or _
- Clauses and clause categories – Be unique within the product line
- Clause terms – Be unique within the parent clause
- Drop-down list entries – Can start with a lowercase letter or digit

Short names cannot contain spaces.

Avoid using similar names within the same clause term. If any of the characters \$, /, or % are used in the short name for a drop-down or package option, they are replaced by an empty string "" when the corresponding code identifiers are generated. This replacement may result in a duplicated name. For example, "100%" and "100\$" are both mapped to "100" during the code generation process.

Short Name fields have character limits that are enforced in the user interface. If you import a mind map or APD template where short names exceed the character limits, the short names are truncated.

Note: The combined length of the Clause and Clause Term names must not exceed 47 characters. The maximum length is 50 if you include the prefix. For example, if a coverage name is 30 characters long, the coverage term name must not exceed 17 characters. APD App does not provide a warning for this limitation, but a warning is raised in the startup logging for PolicyCenter.

The character limits for short names are:

Fields	Limit	Description
Clauses	45	
Clause categories	45	
Clause terms	30	
Risk objects	22	
Exposures	22	
Attributes or fields	30	
Drop-down list values	45	
Product line	22	On the Product Definition screen, PolicyLine Subtype , excluding prefix (*PD)

Words to avoid

The short name cannot be one of the following keywords. The check is case-insensitive.

- Java keywords
- Gosu keywords

- The following SQL keywords:
 - CAST
 - TYPE
 - USE

The name can include these words. For example, `abstract` is a Java and Gosu keyword, so the short name cannot be `ABSTRACT`, but can be `AbstractPhrase`.

Avoid using the short name of a PolicyCenter interface.

Words to avoid for exposures and coverables

For exposures and coverables, field short names cannot match certain property names on PolicyCenter entities. Coverables are risk objects that have coverages, including product lines. Using one of these words results in a warning, but does not block you from completing code generation. Guidewire recommends that you change the name so that no warning is issued.

On exposures and coverables, including the product line, do not use these words as field short names:

- | | |
|-----------------|------------------|
| • Changed | • ExpirationDate |
| • Class | • New |
| • DisplayName | • Owner |
| • EffectiveDate | • TYPE |

On coverables, including the product line, do not use this word as field short names:

- State

On the product line, do not use these words as field short names:

- AdditionalInsureds
- BaseState
- Branch

Note: These are the words to avoid that you are most likely to use. This is not a complete list.

Add metadata

About this task

Fill in all fields that are required to generate the product. In Advanced Product Designer, these fields are marked with * or appear under **Required for Install**.

Note:

Do not add prefixes to codes that identify the line or risk object. The build process will do this for you. These codes appear under **Required for Install**.

Procedure

1. In Advanced Product Designer, navigate to a product.
2. Click **Edit Product** and fill in all required fields on the **Element details** panel.
3. Select the product line, click **Edit Product Line**, and fill in all required fields.
4. Navigate through all elements of the product model, select ... > **View Properties** and fill in required fields on the **Element details** panel.

Review product content

Before generating the product, review all product content.

Procedure

Review all screens and fields for the product and make sure that the information is correct and descriptive.

For example, if you have **Menu label for Generated UI** set to something such as **Risk Objects**, consider changing the name to something more appropriate for the product. Changing **Risk Objects** to **Vehicles** in an auto product would be more descriptive. You can change the **Menu label for Generated UI** in the **Element Details** pane.

Validated fields

The following fields are validated during product generation:

Field	Description
Attributes	Name and Description are required. Typelist name must be unique; two typelists cannot have the same name.
Drop-down list entries	For entries in a drop-down list, Short Name and Description fields on the Drop-Down Definition popup are required. In coverage terms with Option and Package drop-down type, Value is required.
Clauses	For product model clauses, such as coverages, conditions, and exclusions, Short Name and Description are required.
Clause categories	Name and Description must be defined.
Risk objects	For risk objects, including the product line, Description , Menu Label for Generated UI (*PD) , and Type of Risk Object or Product Line Type are required.
Exposures	Description (*PD) , Menu Label (*PD) , and Type are required.
Short names	Short Name (*PD) and Typelist (*PD) must conform to the naming requirements for short names.

In the installed product:

- The **Code** field corresponds to a `CodeIdentifier` property
- The **Menu Label for Generated UI (*PD)** corresponds to a `MenuLabel` property
- The **Product Line Type** or **Type of Risk Object** corresponds to a `TypeName` property

Validate product in Designer mode

A business user working in **Designer** mode in PolicyCenter can validate the product prior to having the product generated by a user with **Developer** mode. In **Designer** mode, the business user can fix some but not all validation errors.

Before you begin

Set **Product Design Mode** to **Designer**.

Procedure

1. Click **Validation Status** to find validation errors.
2. Fix errors, and click **Validation Status** until you have fixed as many errors you can in **Designer** mode.

What to do next

- “Validate product in Developer mode” on page 26

Validate product in Developer mode

In **Developer** mode, the developer can validate the product and fix all validation errors.

Before you begin

- Set **Product Design Mode** to **Developer**.
- “Validate product in Designer mode” on page 25

Procedure

1. In PolicyCenter, go to the **Administration > Product Management** screen.
2. Select a product at the top of the **APD Managed** tab.
The selected product appears in the **Details** tab.
3. On the **Details** tab select **Generate Product Code** and select **Base Product**.

PolicyCenter validates the product model and displays the **Review Product Elements** screen showing the product hierarchy. If there are problems with the product definition, the **Complete Generation** button does not appear in the upper right corner of the screen. Problems are marked with a red X inside a circle in the second column of the table. A red X indicates that there is a error in the current element or one of its children. The element with the error displays an error message. Click > to display the error.

4. If there are problems:
 - a) Click **Cancel Generation**.
 - b) In Advanced Product Designer, fix the problems in the product.
 - c) In PolicyCenter, click **Synchronize With APD**.
 - d) Click **Generate Product Code**.

You cannot proceed until all problems are resolved. There are no problems in the product definition when a check mark appears in the second column of each row in the **Review Product Elements** table. A **Complete Generation** button appears next to **Cancel Generation**.

5. (Recommended) Export your product to an APD template by clicking **Export Template** before generating the product code. You can use this template to create a new product in Advanced Product Designer.
6. (Recommended) Check the APD template into version control. With version control, you can compare versions, and restore previous versions, if necessary.

Note: If you drop the database before the final build is complete, you will lose the product changes you have made.

What to do next

- “Move product” on page 26

Move product

If the product is not in the PolicyCenter instance where it will be generated, deployed, and tested, move it to that instance.

Before you begin

- Set **Product Design Mode** to **Designer** or above.

- “Validate product in Developer mode” on page 26

Procedure

1. In PolicyCenter, go to the **Administration > Product Management** screen.
2. Select the product on the **APD Managed** tab.
3. On the **Details** tab, select **Export Template**.
The APD template is saved on the computer where PolicyCenter is installed.
4. In the PolicyCenter instance where you will generate and deploy the product, go to the **Administration > Product Management** screen.
5. In the **APD Managed** tab, select **Import From Template** and import the APD template.

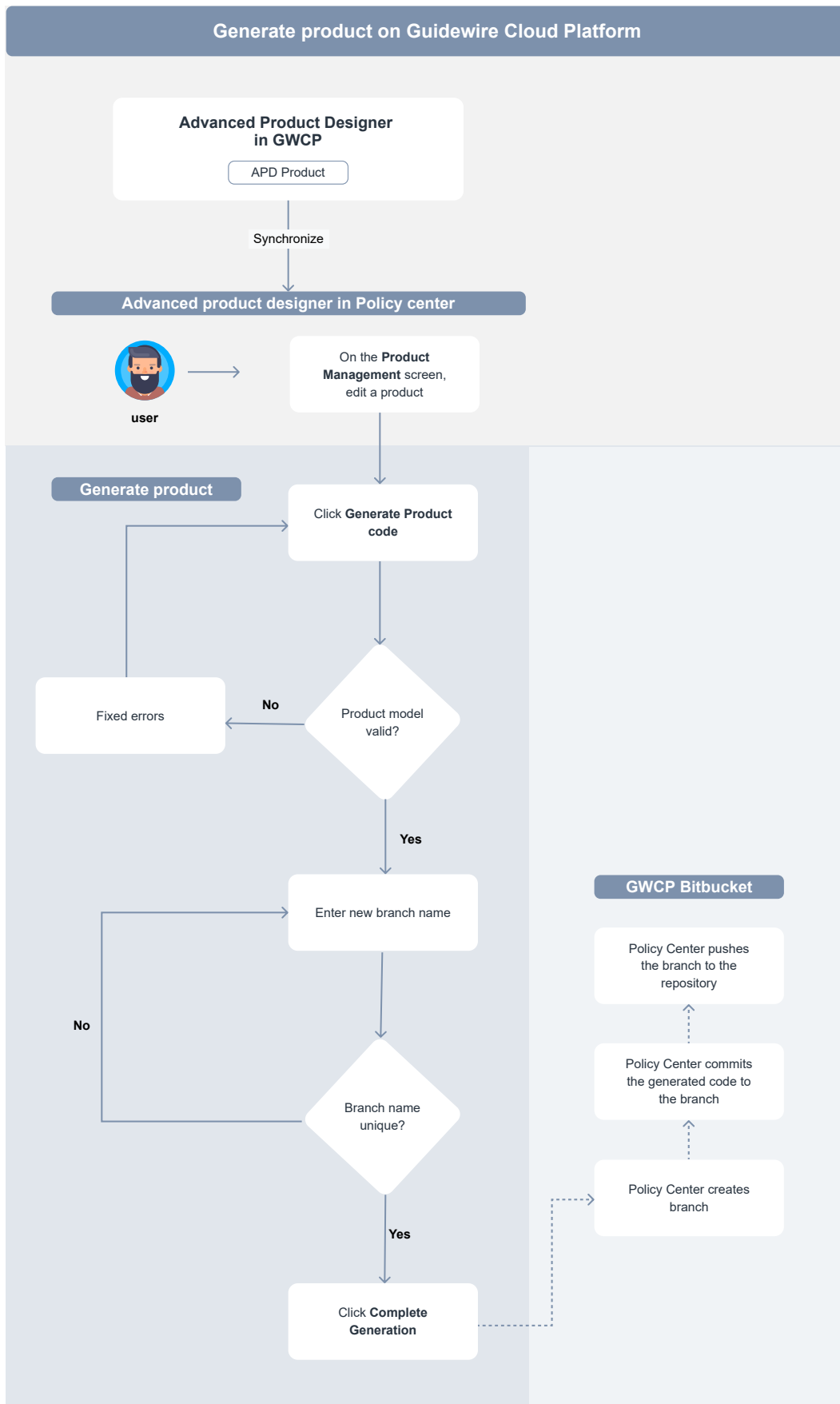
What to do next

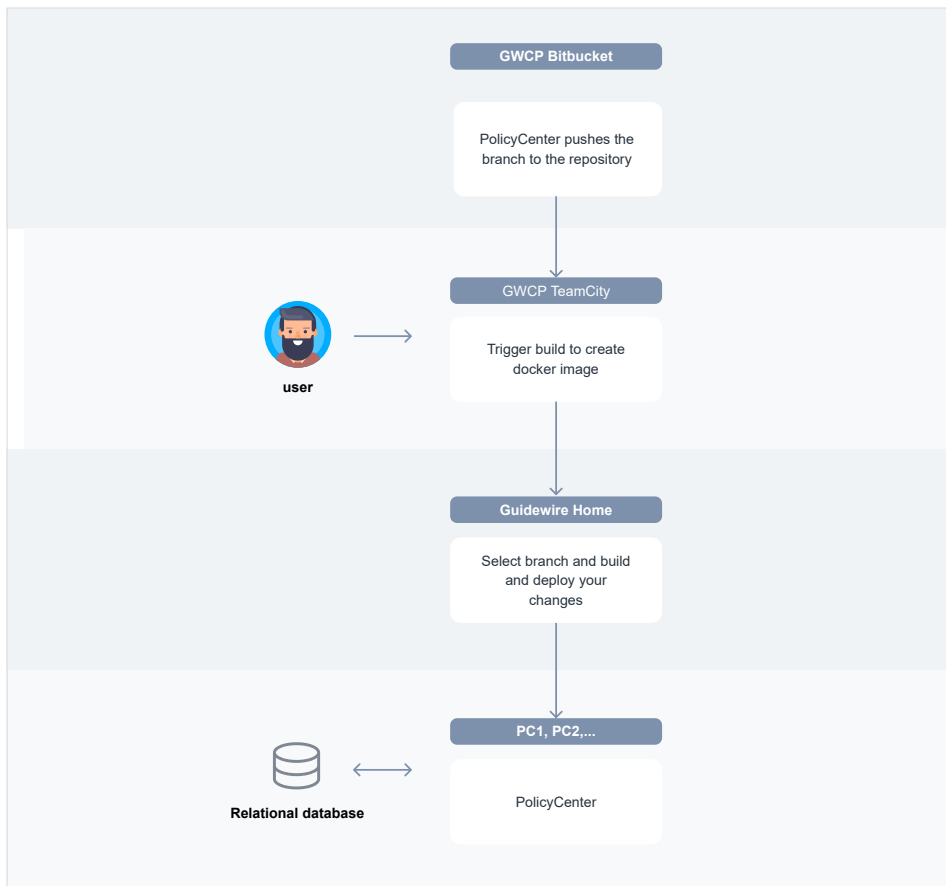
- “Deploy product on Guidewire Cloud” on page 31

Generating and deploying a product

You are ready to finalize a product when you have validated and generated the product, and are ready to deploy the product onto the Guidewire Cloud.

The following illustrations show the process for generating and deploying a product onto Guidewire Cloud to various PolicyCenter instances, such as PC1, PC2, and so on.





Edit PolicyCenter settings

To edit PolicyCenter settings:

1. On the **Product Management** screen, select the product and click **View Product**.
2. On the **Product Definition** screen, click **Edit PC Settings**.
 - **Enabled For REST API Controls** whether visualized or finalized submissions are created.
 - Set to **No** for finalized product.
 - Set to **Yes** for visualized product.
 - **App Event Aware** Application Events are a Guidewire Cloud capability that simplify integrations. Events allows downstream systems to subscribe to business events without coding.
 - Choose **Yes** to generate a product that is compatible with Application Events.
 - If you choose **No**, then Application Events cannot be used with this product.

Alternatively, you can set **App Event Aware** in APD App by displaying the **Element Details** for the policy line and, in **Info**, checking the **App Event Aware** checkbox.

Define Gosu package for product line files

Before you start generating products, define the Gosu package that contains the Gosu classes for your product lines.

The `APDProductLinesGosuPackage` script parameter defines the package in which PolicyCenter generates Gosu files for your product lines. The package is created in `gsrsrc`, and the default value is `ext.lob`. You can change the package name to a name appropriate for your implementation, such as `xyzcorp` or `ext.xyzcorp`.

1. Access the `APDProductLinesGosuPackage` parameter in one of the following ways:
 - a. If PolicyCenter is not running and the database is empty, edit `ScriptParameters.xml` in Guidewire Studio.
 - b. Go to **Administration > Utilities > Script Parameters** in PolicyCenter.
2. Change the value of `APDProductLinesGosuPackage` from `ext.lob` to a name appropriate for your implementation, such as `xyzcorp`. Gosu classes for your product line are created in the `xyzcorp` package. In Guidewire Studio, you can access these files in `configuration/gsrc/xyzcorp`.
3. If you have previously generated product lines, move the Gosu classes for those product lines from the old to the new package.
 - a. Stop PolicyCenter if it is running.
 - b. In Guidewire Studio or in the file system move the Gosu classes from the old to the new package.
 - c. Restart PolicyCenter.

Generate product code

In PolicyCenter, on the **Product Management** screen, generate product code by clicking **Generate Product Code**.

Generate Options

From the **Review Product Elements** screen, in **Generate Options**, you can select:

Base Product

Generate PolicyCenter product artifacts only. These are the data model, product model, and supporting Gosu code.

Rating Files

Generate rating files.

Page Configuration Files

Generate page configuration files.

System APIs - Code

Generate the system APIs.

System APIs - Tests

Generate some API unit tests for the product that you can maintain.

Extensions

Generate additional registered extensions only.

For each element you can select **Default Directory** or **Do Not Generate**. For **Rating Files** and **Page Configuration Files**, you can also select **Temporary Directory**. It is not possible to generate **Rating Files** and **Page Configuration Files** if **Base Product** generation is disabled.

The `migratedGsrcNotCode` directory

In the following two cases where generation is configured to use a temporary directory, files are generated into the `migratedGsrcNotCode` directory:

- You choose to generate Page Configuration Files or Rating Files into a temporary directory.
- If there is a class, for example an adapter, that is not generated, the class is stored in a temporary directory. For example, if there are Coverable entities that already implement the CoverableAdapter interface using custom classes, APD does not overwrite those classes. Instead the code is generated into the `migratedGsrcNotCode` directory so that you can see what APD would have generated.

Code generation in a local development environment

In a local development environment, not a cloud-deployed environment, you must generate code for one product at a time. Files are created locally. You must handle any git operations.

Code generation on a deployed planet

On a deployed planet, you can generate code for a single product or for multiple products:

- For code generation for a single product, the repository is cloned. The code is then generated and pushed to git and the directory is cleaned.
- For code generation for multiple products, the code for each product is generated sequentially, without cleaning the directory after each code generation, so the branch has all changes without requiring a manual merge.

Bitbucket authentication

When performing code generation from a deployed PolicyCenter, Bitbucket authentication uses SSH by default. Use these two property service settings to enable SSH: `pc.git.ssh.url` and `pc.codegen.git.ssh.private_key`. If the SSH URL setting does not exist or is empty, HTTPS authentication is used instead. If the tenant does not have SSH enabled for Bitbucket or the SSH keys are not correct, a Git cloning error is logged.

Deploy product on Guidewire Cloud

Use these steps to finalize a product from PolicyCenter running on Guidewire Cloud.

Before you begin

Complete the tasks in “Code generation prerequisites” and “Generate product code”.

Procedure

1. In Guidewire Cloud, go to PolicyCenter.
2. Go to the **Product Management** screen and synchronize the APD App product with PolicyCenter.
3. Select the product and then select **Generate Product Code**.
The code repository is initialized, and PolicyCenter displays a **Branch Name** field on the **Review Product Elements** screen. The **Branch Name** field only appears in PolicyCenter running on Guidewire Cloud. A **Complete Generation** button appears next to **Cancel Generation**.
4. Review or select the **Generate Options**.
For information on the generate options, see “Generate product code” on page 30.
5. In **Branch Name**, enter a branch name. The branch name must be unique within the Bitbucket repository. PolicyCenter displays an error message if you enter a branch name that is not unique.
6. Click **Complete Generation**.
Complete Generation appears when product validation passes and you have entered a valid branch name.
PolicyCenter generates code, creates a new branch, commits the generated code to that branch, and then pushes the branch to the Bitbucket repository.
7. Navigate to Guidewire Home.
For more information about how to perform this and the following steps, see *Cloud Platform*.
8. In TeamCity, trigger a build that creates a docker image of the application.
9. From Guidewire Home, deploy the docker image, selecting the branch and build number.
10. Deploy to PolicyCenter instances, such as PC1, PC2, and so on.

Regenerating product code

After you generate the product, you can enhance the installed product using the development tools provided by PolicyCenter, such as Guidewire Studio and the Cloud API. You can create underwriting rules, rating, and forms management. You can integrate with other applications, and make other changes to meet your needs. You incorporate changes made to the APD App product by regenerating it and merging changes.

As you continue product development, you can change the product in product editions and by modifying the product structure.

Making deletions in the visualized product

After completing product code generation, if you make deletions in the visualized product then regenerate the product, the deletions are not removed from the installed product. You must delete them manually in Product Designer or Guidewire Studio. These deletions could be the result of reimporting the mind map or APD template, or of deleting something from the product in Advanced Product Designer App.

Regenerating the product

After generating your product, you can continue product development in Advanced Product Designer App while simultaneously making changes to the product in Guidewire Studio and Product Designer.

If a feature is provided in Advanced Product Designer App, then always use Advanced Product Designer App to develop the feature. Do not use Product Designer or Studio to add product features that Advanced Product Designer App provides. See “Updating insurance products” on page 41.

To update the installed product, you need to regenerate it. When you click **Generate Product Code**, Advanced Product Designer displays a warning that lists the modified files that will be overwritten if you proceed. Before completing the generation, make a copy of these files so that you can merge your changes after regenerating.

For each type of file, the following table shows whether you are warned before regenerating.

Type of files	Warn ?	Description
“System files” on page 33		No warning if file is modified.
“Resources” on page 34		No warning if file is modified. The files <code>yyycostdef.xml</code> , <code>XXXDiffTree.xml</code> , and <code>YYYDiffTree.xml</code> are exceptions.
“Product model” on page 34		No warning if file is modified.
“Policy line patterns” on page 35		No warning if file is modified.
“Risk object entities” on page 37	•	If you can, make all changes to the object in Advanced Product Designer App. When you regenerate the product, you can accept the regenerated version of the object, and do not need to merge it with the older version. If that is not possible, you can make changes in an extension to the object. You have to add the extension manually in the file system. Although you cannot add the extension in Studio, you can edit it in Studio after creating it.
“Clause entities” on page 38	•	
“Schedule entities” on page 38	•	
“Typelists” on page 39	•	For each typelist, maintain it either in a particular Advanced Product Designer App product or in Studio. Do not modify the typelist in more than one place.
“Display names” on page 39	•	Make all changes to display strings in Advanced Product Designer App so that there is no need to merge changes when you regenerate.
“PCF files” on page 39	•	Save a copy before generating, and merge changes.
“Gosu classes” on page 40	•	When you generate the product, Advanced Product Designer App creates a base class and an implementation class in the <code>PolicyCenter_installation/modules/configuration/gsrc</code> directory. The base class is created in the <code>gw.lob</code> package and has the suffix <code>Base</code>. The implementation class is created in the package defined by the <code>APDProductLinesGosuPackage</code> script parameter. The default is <code>ext.lob</code>. To avoid triggering warnings when you regenerate the product, make your changes in the implementation class, and do not modify the base class. You will receive warnings for changes to base classes, but no warnings for changes to implementation classes. When you regenerate, the implementation class is not affected; it is only generated if it does not exist.

Type of files	Warn ?	Description
		Do not modify any classes that are prefixed with APD or any package with apd in the name, for example gw.apd. For gw.web.rules.APDRulesHelper.gs, create an extension of this class, for example APDRulesHelper_Ext, and add your methods into it. These methods are used by customized PCFs.
		For enhancement classes, <i>YYYRiskEnhancement.gs</i> and <i>YYYExposureEnhancement.gs</i> , make your changes directly in the enhancement and merge changes.

Note: Some files are generated only once. You can make changes, and your changes are not overwritten when you regenerate. These exceptions are noted in the topics describing the file types.

Merging changes in installed product

When merging changes to files, do a three-way merge between:

- File as originally generated by Advanced Product Designer App
- Modifications that you made to the file
- File as generated this time by Advanced Product Designer App

Installed product files

Generating a product creates files in the *PolicyCenter_installation/modules/configuration/config* directory of PolicyCenter.

The files are listed below. The following variables are used in the file names:

- *xxx* and *xxx* – The eight-character product abbreviation in upper and lower case
- *yyy* and *yyy* – The eight-character product line abbreviation in upper and lower case
- *ProdId* – Product identifier
- *LineId* – Line identifier
- *Line* – Short name of the product line
- *Clause* – Short name of the clause
- *TypeList* – Short name of the typelist
- *Risk* – Short name of the risk object

System files

Information about the product line appears at the end of the following files. Paths are relative to *PolicyCenter_installation/modules/configuration/config*.

When regenerating the product, you are not warned about modified files.

Subdirectory	File name	Description
locale	display.properties	Display labels for screens. Generates labels for all languages defined by the product. In Studio, if you add a display label that is not generated, the display key remains intact after regenerating the product.
	productmodel.display.properties	Display labels for user interface fields for all languages defined by the product.
	typelist.properties	For all languages defined by the product:

Subdirectory	File name	Description
		- TypeKey. InstalledPolicyLine.XXX – Type key for policy line. - TypeKeyDescription. InstalledPolicyLine.XXX – Type key for description.
lookuptables	lookuptables.xml	Lookup table for all product model patterns.

Resources

Product generation adds files to the resources directory under *PolicyCenter_installation/modules/configuration/config*.

When regenerating the product, you are not warned about modified files. The files *yyycostdef.xml*, *XXDiffTree.xml*, and *YYDiffTree.xml* are exceptions.

Subdirectory	Directory or File name	Description
	systables.xml	Cost definition for the product line: <pre><FileDefinition Name="XXXcostdef.xml" Priority="0"> <Entity Type="XXXCostDefinition"/> </FileDefinition></pre>
diff	XXDiffTree.xml	Before regenerating the product, you will receive a warning about modified files that will be overwritten if you proceed.
productmodel	policylinepatterns/ <i>LineId</i> products/ <i>ProdId</i>	
systables	costcode_ext.xml	
	yyycostdef.xml	Before regenerating the product, you will receive a warning about modified files that will be overwritten if you proceed.
	underwriting_companies.xml	For each jurisdiction: <pre><LicensedState public-id="ProdId:Jurisdiction"></pre>

Product model

Product model files are in the resources/productmodel/products directory under *PolicyCenter_installation/modules/configuration/config*.

When regenerating the product, you are not warned about modified files.

File name	Description
<i>ProdId/ProdId.xml</i>	These files are generated once. You can make changes, and your changes will not be overwritten when you regenerate.
<i>ProdId/ProdId-lookups.xml</i>	

Update availability in Product Designer

If possible update availability in Advanced Product Designer App. If Advanced Product Designer App does not support a particular type of availability, you can make updates directly in Product Designer.

The first column of the table describes the update you make in Product Designer. The second column describes what happens in Studio when you commit the change in Product Designer. The third column describes what happens when you regenerate the product in Advanced Product Designer App.

Product Designer	Studio	After regenerating product
Availability on Product	Availability criteria is updated in <i>ProdId/ProdId-lookups.xml</i> . Availability criteria for a jurisdiction is updated in <i>ProdId-lookups.xml</i> in the <i>ProdId/jurisdictions/XX/coveragepatterns</i> directory, where <i>XX</i> is the jurisdiction.	Updates in all <i>ProdId-lookups.xml</i> files remain intact.

Update offerings in Product Designer

Advanced Product Designer App does not support offerings. You can specify offerings for your product in Product Designer.

Product Designer	Studio	After regenerating product
Add offering	<i>XXXOffering.xml</i> and <i>XXXOffering-lookups.xml</i> created in the <i>ProdId/offerings</i> directory, where <i>XXX</i> is the name of the offering.	<i>XXXOffering.xml</i> and <i>XXXOffering-lookups.xml</i> files remain intact. Overwritten – Offering disappears from <i>productmodel.display.properties</i> without warning.

Update question sets in Product Designer

Advanced Product Designer App does not support question sets. You can specify question sets for your product in Product Designer.

The first column of the table describes the update you make in Product Designer. The second column describes what happens in Studio when you commit the change in Product Designer. The third column describes what happens when you regenerate the product in Advanced Product Designer App.

Product Designer	Studio	After regenerating product
Add question set with question	Question set added to <i>ProdId/ProdId.xml</i> . <i>QuestionSetName.xml</i> and <i>QuestionSetName-lookups.xml</i> created in <i>productmodel/questionsets</i> .	Files remain intact.

Policy line patterns

Policy line patterns are in *resources/productmodel/policylinepatterns/Line* under *PolicyCenter_installation/modules/configuration/config*. The policy line corresponds to the product line in Advanced Product Designer App.

When regenerating the product, you are not warned about modified files.

File name	Description
<i>Line.xml</i>	These files are generated once. You can make changes, and your changes will not be overwritten when you regenerate.
<i>Line.properties</i>	
<i>Line-lookups.xml</i>	

Update categories in Product Designer

Guidewire recommends that you update categories in Advanced Product Designer App not Product Designer. In Advanced Product Designer App, these are *clause categories*. This topic describes the consequences of updating categories in Product Designer.

The first column of the table describes the update you make in Product Designer. The second column describes what happens in Studio when you commit the change in Product Designer. The third column describes what happens when you regenerate the product in Advanced Product Designer App.

Product Designer	Studio	After regenerating product
Add new category	Category is added to <i>Line.xml</i> in the <i>policylinepatterns</i> directory. Display string for category is added to <i>productmodel.display.properties</i> .	Overwritten – Category disappears from both files without warning.
Modify category	Category is modified in <i>Line.xml</i> in the <i>policylinepatterns</i> directory. Display string for category is modified in <i>productmodel.display.properties</i> .	Overwritten – Your modifications to both files disappear without warning.
Remove category	Category is removed from <i>Line.xml</i> in the <i>policylinepatterns</i> directory. Display string for category is removed from <i>productmodel.display.properties</i> .	Overwritten – The removed category reappears in both files without warning.

Update clauses in Product Designer

Guidewire recommends that you update clauses in Advanced Product Designer App not Product Designer. This topic describes the consequences of updating clauses in Product Designer.

Clause patterns are in *resources/productmodel/policylinepatterns/Line/coveragepatterns*. This directory includes patterns for coverages, conditions, and exclusions.

File name
<i>YYYClause.xml</i>
<i>YYYClause-lookups.xml</i>

The first column of the table describes the update you make in Product Designer. The second column describes what happens in Studio when you commit the change in Product Designer. The third column describes what happens when you regenerate the product in Advanced Product Designer App.

Product Designer	Studio	After regenerating product
Add clause	<i>YYYClause.xml</i> and <i>YYYClause-lookups.xml</i> files are added in the <i>coveragepatterns</i> directory. Display string for clause is added to <i>productmodel.display.properties</i> .	<i>YYYClause.xml</i> and <i>YYYClause-lookups.xml</i> files remain intact. Overwritten – Clause disappears from <i>productmodel.display.properties</i> without warning.
Remove clause	<i>YYYClause.xml</i> and <i>YYYClause-lookups.xml</i> files are removed from the <i>coveragepatterns</i> directory. Display string for clause is removed from <i>productmodel.display.properties</i> .	Overwritten – <i>YYYClause.xml</i> and <i>YYYClause-lookups.xml</i> files reappear after generate without warning. Overwritten – Clause reappears in <i>productmodel.display.properties</i> without warning.
Add term to existing clause	Term is added to <i>YYYClause.xml</i> and <i>YYYClause-lookups.xml</i> files in the <i>coveragepatterns</i> directory.	Overwritten – Term disappears from <i>YYYClause.xml</i> without warning. Term remains in <i>YYYClause-lookups.xml</i> .

Product Designer	Studio	After regenerating product
	Display string for term is added to <code>productmodel.display.properties</code> .	Overwritten – Clause disappears from <code>productmodel.display.properties</code> without warning.
Remove term from clause	Term is removed from <code>YYYClause.xml</code> and <code>YYYClause-lookups.xml</code> files in the <code>coveragepatterns</code> directory. Display string for term is removed from <code>productmodel.display.properties</code> .	Overwritten – Term reappears in <code>YYYClause.xml</code> and <code>YYYClause-lookups.xml</code> without warning. Overwritten – Term reappears in <code>productmodel.display.properties</code> without warning.

Update availability in Product Designer

If possible, define availability for the product in Advanced Product Designer App. If you have requirements for usage and availability that cannot be specified using Advanced Product Designer App, you can use the availability features that Product Designer offers. For more information, see “Updating insurance products” on page 41.

Availability scripts, lookup tables, and enabling legacy coverage are not supported in Advanced Product Designer App. You can enter these for your product line in Product Designer.

The first column of the table describes the update you make in Product Designer. The second column describes what happens in Studio when you commit the change in Product Designer. The third column describes what happens when you regenerate the product in Advanced Product Designer App.

Product Designer	Studio	After regenerating product
Lookup table (availability criteria that you define at the top of the Availability screen).	Availability criteria is updated in <code>YYYClause-lookups.xml</code> in the <code>coveragepatterns</code> directory. Availability criteria for a jurisdiction is updated in <code>YYYClause-lookups.xml</code> in the <code>Line/jurisdictions/XX/coveragepatterns</code> directory, where <code>XX</code> is the jurisdiction.	Updates in all <code>YYYClause-lookups.xml</code> files remain intact.
Availability script	Script is updated in <code>YYYClause.xml</code> in the <code>coveragepatterns</code> directory.	For clauses, availability script updates remain in <code>YYYClause.xml</code> . Overwritten – Availability script is overwritten in clause terms. You are warned before the file is overwritten.
Enabling legacy coverage (randomly choose if covered object at line or risk)	Criteria is updated in <code>YYYClause-grandfathering.xml</code> in the <code>Line/jurisdictions/XX/coveragepatterns</code> directory, where <code>XX</code> is the jurisdiction.	Updates in <code>YYYClause-grandfathering.xml</code> remain intact.

Risk object entities

Entities for risk objects are in `extensions/entity` directory under `PolicyCenter_installation/modules/configuration/config`.

Before regenerating the product, you will receive a warning about modified files that will be overwritten if you proceed.

If you can, make all changes to the object in Advanced Product Designer App. When you regenerate the product, you can accept the regenerated version of the object, and do not need to merge it with the older version. If that is not possible, you can make changes in an extension to the object. You have to add the extension manually in the file system. Although you cannot add the extension in Studio, you can edit it in Studio after creating it.

File name	Description
YYCost.eti	For each product line, there is a cost on the product line. These files are generated once. You can make changes, and your changes will not be overwritten when you regenerate.
YYCostDefinition.eti	
YYTransaction.eti	These files are generated once. You can make changes, and your changes will not be overwritten when you regenerate.
YYRisk.eti	An entity for each risk object, including the product line. These files are generated once. You can make changes, and your changes will not be overwritten when you regenerate.
YYRiskCost.eti	A cost entity for each risk object.
YYExposure.eti	An entity for each exposure.
YYLineId.eti	An entity for the product line.

Clause entities

Entities for clauses are in the extensions/entity directory under *PolicyCenter_installation/modules/configuration/config*. Clause types are coverages, conditions, and exclusions.

Before regenerating the product, you will receive a warning about modified files that will be overwritten if you proceed.

If you can, make all changes to the object in Advanced Product Designer App. When you regenerate the product, you can accept the regenerated version of the object, and do not need to merge it with the older version. If that is not possible, you can make changes in an extension to the object. You have to add the extension manually in the file system. Although you cannot add the extension in Studio, you can edit it in Studio after creating it.

File name	Description
YYRiskCov.eti	An entity for each clause category on the risk object.
YYRiskCond.eti	
YYRiskExcl.eti	
YYRiskCovCost.eti	A cost entity for each clause type.
YYRiskCovLookup.eti	These files are generated once. You can make changes, and your changes will not be overwritten when you regenerate.
YYRiskCondLookup.eti	
YYRiskExclLookup.eti	
YYRiskOptLookup.eti	
YYRiskPackLookup.eti	
YYRiskTermLookup.eti	

Schedule entities

Entities for schedules are in the extensions/entity directory under *PolicyCenter_installation/modules/configuration/config*.

Before regenerating the product, you will receive a warning about modified files that will be overwritten if you proceed.

If you can, make all changes to the object in Advanced Product Designer App. When you regenerate the product, you can accept the regenerated version of the object, and do not need to merge it with the older version. If that is not possible, you can make changes in an extension to the object. You have to add the extension manually in the file system. Although you cannot add the extension in Studio, you can edit it in Studio after creating it.

File name

YYYRiskSchCovItemCov.eti

YYYRiskScheduleCov.eti

YYYRiskScheduleCovItem.eti

YYYScheduledItem.eti

Display names

Language display strings for each entity are in the `displaynames` directory under `PolicyCenter_installation/modules/configuration/config`. The file names are:

- `YYYEntity.Lang-code`

For example, `PETDogCov.en` or `PETDogCov.fr`.

Before regenerating the product, you will receive a warning about modified files that will be overwritten if you proceed.

Make all changes to display strings in Advanced Product Designer App so that there is no need to merge changes when you regenerate.

In Studio, if you add a display name that is not generated, the display name remains intact after regenerating the product.

Costs and transactions

The following files are generated once. You can make changes, and your changes will not be overwritten when you regenerate.

- `YYVCost.Lang-code`
- `YYYTransactionLang-code`

Typelists

Typelists in the `extensions/typelist` directory under `PolicyCenter_installation/modules/configuration/config`.

Before regenerating the product, you will receive a warning about modified files that will be overwritten if you proceed.

For each typelist, maintain it either in a particular Advanced Product Designer App product or in Studio. Do not modify the typelist in more than one place.

File name	Description
<code>Typelist.tti</code>	Typelist for each drop-down list defined by the product line.
<code>InstalledPolicyLine.YYY.ttx</code>	Adds this policy line to the <code>InstalledPolicyLine</code> typelist.

PCF files

Subdirectories for PCF files are in the `web/pcf/line, xxx` and `yyy` directories. Paths are relative to `PolicyCenter_installation/modules/configuration/config`.

Before regenerating the product, you will receive a warning about modified files that will be overwritten if you proceed.

PCF files that you add are not overwritten when you regenerate the product.

For the product, the directory contains the following file:

Subdirectory	File name
policyfile	PolicyMenuItemSet. <i>ProdId</i> .pcf

For the product line, the directory contains the following files:

Subdirectory	File name
job	LineWizardStepSet. <i>ProdId</i> .pcf
policy	YYYLineIdPanelSet.pcf
	YYYLineIdScreen.pcf
policyfile	PolicyFile_YYYLineId.pcf
	LineIdLinks.pcf

For each risk object, the directory contains the following files:

Subdirectory	File name
policy	YYYRiskListPanelSet.pcf
	YYYRiskPanelSet.pcf
	YYYRiskPopup.pcf
	YYYRiskScreen.pcf
policyfile	PolicyFile_YYYRisk.pcf
	PolicyFile_YYYRiskScreen.pcf

For each exposure, the directory contains the following files:

Subdirectory	File name
policy	YYYExposureListPanelSet.pcf

Gosu classes

When you generate the product, Advanced Product Designer App creates a base class and an implementation class in the *PolicyCenter_installation/modules/configuration/gsrc* directory. The base class is created in the *gw.lob* package and has the suffix *Base*. The implementation class is created in the package defined by the *APDProductLinesGosuPackage* script parameter. The default is *ext.lob*.

To avoid triggering warnings when you regenerate the product, make your changes in the implementation class, and do not modify the base class. You will receive warnings for changes to base classes, but no warnings for changes to implementation classes. When you regenerate, the implementation class is not affected; it is only generated if it does not exist.

Do not modify any classes that are prefixed with *APD* or any package with *apd* in the name, for example *gw.apd*. For *gw.web.rules.APDRulesHelper.gs*, create an extension of this class, for example *APDRulesHelper_Ext*, and add your methods into it. These methods are used by customized PCFs.

For enhancement classes, *YYYRiskEnhancement.gs* and *YYYExposureEnhancement.gs*, make your changes directly in the enhancement and merge changes.

Variations in installed products

Displaying Risk Details in installed product

In the installed product, the **Risk Details** screen may display differently than in the visualized product.

In the installed product, the **Risk Details** screen layout uses the following rules for displaying risk objects, using the first match:

Condition	Display
Risk object has more than eight fields	Display fields in two columns.
Risk object has eight or fewer fields	
Only one type of child risk object	Display fields in two columns and list risk objects underneath the fields.
More than one type of child risk object or no risk objects, and exposure details are defined	Display a list of exposures underneath the fields. For example, display a list of exposures in a Workers' Compensation policy.
No exposures or child risk object	In first column, display fields. In second column, display a list of coverages.
No fields	Depending upon what exists, display: • Risk objects if there is just a single child risk object type, or • Exposures, or • Coverages

Risk objects and coverables

In the installed product, risk objects with coverages become **Coverable** entities. If a risk object has no coverages, then it is not a **Coverable**.

Updating insurance products

Advanced Product Designer App is an excellent way to start developing the product model for your insurance product. However, product development is an iterative process and the tool that you use to update the insurance product depends on the required changes.

Updates in Advanced Product Designer App

Make the following types of product updates in Advanced Product Designer App:

Structural Elements

Always add new structural elements, such as clauses, risk objects, modifiers, or attributes, in Advanced Product Designer App rather than adding them in Guidewire Studio or Product Designer. If you add structural elements in Guidewire Studio or Product Designer, you cannot alter them in the product in Advanced Product Designer App.

Product Model Properties

Specify minimum and maximum values and availability in Advanced Product Designer App. If the mind map is your source of truth, add these elements in the mind map, and update the product with the mind map.

Product Editions

Use product editions to introduce changes to the product after a product has gone into production. Typically, products start out with only the Base edition. Create additional editions if there is a filing or other business reason to change the product as of a certain effective date.

Updates in Product Designer

Product Designer provides additional features that you can use to update your insurance product such as:

Availability scripts

Gosu scripts that can capture complex logic. For example, use a script to perform an action to determine when a clause or action is removed.

Enabling legacy coverage

Continue to offer a coverage or other pattern to existing customers, even if the pattern is not otherwise available.

Offerings

Enable or disable product model patterns for different sales or customer situations.

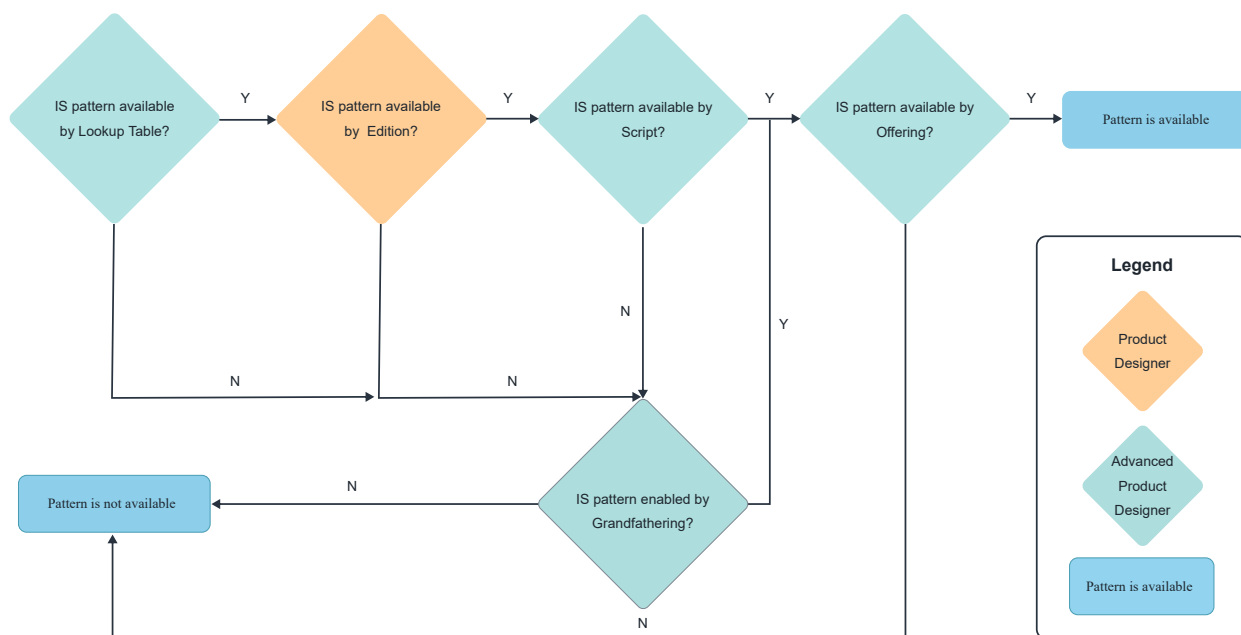
Availability lookup tables

Specify whether a given pattern is available based on date, jurisdiction, industry code, transaction type, underwriting company, among others.

Note: When possible, use the availability features provided in Advanced Product Designer App. Only use Product Designer for availability requirements that Advanced Product Designer App does not provide.

Determining the availability of a pattern

If you use a combination of availability features from Advanced Product Designer App and Product Designer, availability is determined as follows:



Next steps

After you update the product, you can:

- Visualize your insurance product. See “Visualizing APD App products” on page 11.
- Finalize and generate your insurance product. See “Finalizing and generating the product” on page 21.

Related information

- *Product Model*